

Farhan Aziz Harahap.

Software Developer — Web Applications
& Python Automation

Informatics student at Universitas Putera Batam. I build database-backed web apps and automation tools — tested, versioned, and continuously integrated. Four selected projects inside, with full case studies.

EMAIL

ggfarhan07@gmail.com

LINKEDIN

[linkedin.com/in/farhan-aziz-harahap-728b421a5](https://www.linkedin.com/in/farhan-aziz-harahap-728b421a5)

GITHUB

github.com/sakeeeeeeeee

PORTFOLIO SITE

farhan-aziz-harahap.vercel.app

ABOUT ME

Turning ideas into **useful software**.

I'm an informatics student who enjoys turning complex problems into simple, thoughtful products — from Laravel web apps to Python automation. I care about doing things properly: tested, versioned, and continuously integrated.

My biggest project, **Hotelaz**, is a complete hotel management system covered by 18 automated feature tests running on CI. I'm now preparing for the Apple Developer Academy @ Infinite Learning (Cohort 2027), where I plan to bring the same engineering discipline to Apple platforms.

SKILLS & TOOLS

Programming

PHP · Python · JavaScript · SQL

Web Development

Laravel · React · Tailwind CSS · HTML/CSS

Tools & Practices

Git & GitHub · CI (GitHub Actions) · PHPUnit · Vite

JOURNEY

2023

Started learning programming

Began my informatics degree at Universitas Putera Batam and wrote my first HTML pages and simple programs.

2025

Built my first serious projects

Shipped my first database-backed web apps in PHP, then moved to Laravel with Hotelaz — a complete hotel management system with tests and CI.

2026

Automation, bots & testing

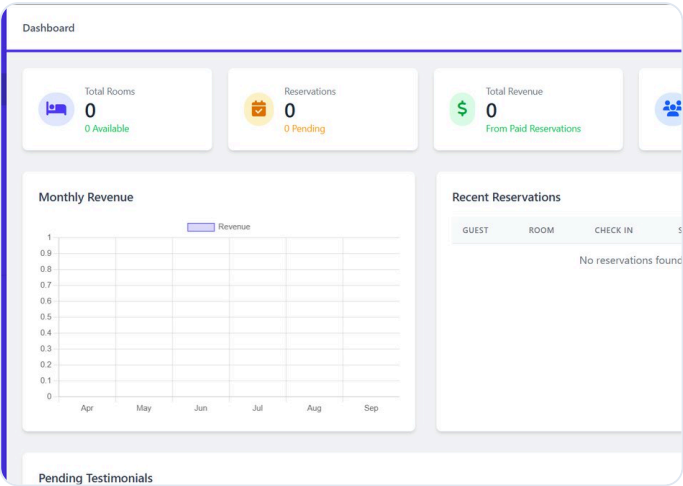
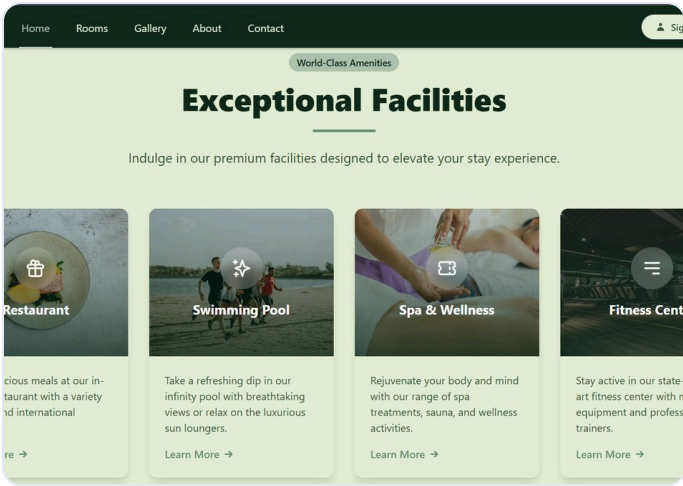
Built a Python trading-signal bot with MetaTrader 5 and Telegram, plus data tooling. Now focused on testing, CI, and software that holds up under real use.

Hotelaz — Hotel Management System

Laravel

A full-featured hotel management web app: room browsing, reservations with overbooking protection, payment-proof upload, email notifications, and an admin dashboard with revenue statistics.

- PHP
- Blade
- Tailwind CSS
- MySQL



PROBLEM

Small hotel operations rely on scattered tools — paper records or spreadsheets — to manage rooms, bookings, and payments. This causes double-bookings, lost reservations, and no clear picture of revenue.

APPROACH

Started with the data model (rooms, room units, reservations, testimonials, galleries), then built the guest booking flow, then the admin panel. Iterated with feature tests after the core worked.

SOLUTION

Implemented an overlapping date-range query compared against each room's unit quantity — a booking is only accepted if active (non-cancelled) reservations for that range stay below the room count. Each room type tracks individual physical units (RoomUnit) with their own status. SMTP credentials and uploads are handled via validated storage and environment config.

WHAT I LEARNED

Designing correctness-critical queries (overlap detection), structuring a Laravel app with middleware-based role separation, policies for resource ownership, and testing business rules instead of just CRUD.

MY ROLE

Solo developer — database design, backend, frontend UI, and deployment setup.

KEY CHALLENGE

The hardest part was availability logic: preventing overbooking when multiple reservations overlap partially with the requested date range. Also handling file uploads securely and separating guest/admin authorization.

RESULT

18 automated tests (43 assertions) covering the reservation flow, availability logic, authorization, and email delivery — all passing on CI (GitHub Actions). The app sends confirmation emails with a unique booking reference (resi) generated on confirmation.

NEXT STEPS

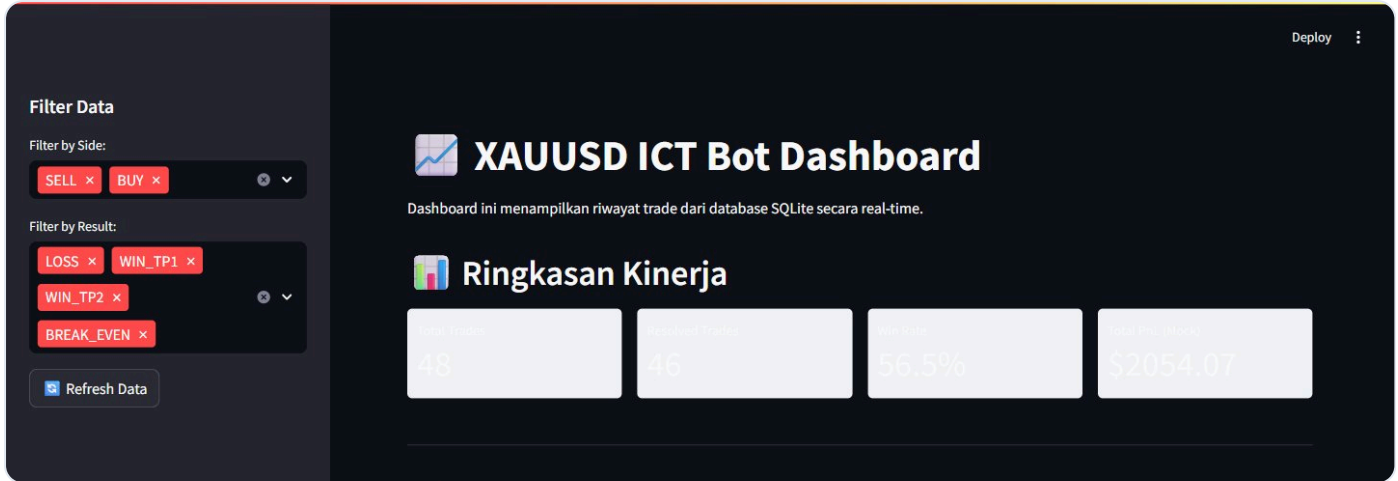
Add a payment gateway integration (the current flow records payment proofs manually), a calendar view for admins, and rate-based pricing on peak dates.

XAUUSD ICT Trading Signal Bot

Python

A rule-based trading signal bot for Gold (XAUUSD) implementing the ICT Silver Bullet strategy: FVG detection, session filters, smart risk management, Telegram integration, and an analytics dashboard.

- MetaTrader 5
- SQLite
- Streamlit
- Telegram Bot



PROBLEM

Manual intraday traders miss setups because monitoring charts during London and New York sessions is impractical, and emotions break consistency. Existing bots are black boxes with no risk validation.

APPROACH

Studied the ICT Silver Bullet methodology first, translated it into deterministic rules (H4 bias → FVG retest → confluence scoring), then layered session filters, an anti-overfitting profile, and walk-forward backtesting to validate parameters before going live.

SOLUTION

Built a Dual-Tier architecture: the core engine runs uninterrupted on the main thread (scanning, FVG detection, SQLite persistence) while the Telegram interface answers commands asynchronously in a background thread. Risk manager computes ATR-based stop loss, enforces R:R bounds, and validates min/max risk. A backtest engine with walk-forward train/test splits and an offline Gemini-analysis script (weekly, manual) complete the loop — LLMs are explicitly excluded from live execution.

WHAT I LEARNED

Translating a discretionary trading methodology into deterministic code, walk-forward testing to fight overfitting, threading and async design for responsive bots, and separating measurable evaluation from execution.

MY ROLE

Solo developer — strategy engine, MT5 integration, risk manager, bot infrastructure, and dashboard.

KEY CHALLENGE

Preventing overfitting was central: a strategy tuned to historical data fails live. Also, running the scan engine and the interactive Telegram interface in one process without blocking each other.

RESULT

A complete pipeline: MT5 data → signal detection → risk validation → Telegram broadcast with charts → outcome tracking in SQLite → Streamlit analytics showing historical win rate. Covered by unit tests for risk and analysis modules.

NEXT STEPS

Expand to multi-symbol scanning, add proper position-sizing from account equity, and publish the backtest reports as static pages.

Twitter/X Keyword Scraper

Python

A Python utility that collects posts matching a keyword with cookie-based account rotation, exporting results to CSV and JSON for analysis.

Twikit

CSV

JSON

```
Microsoft Windows [Version 10.0.22631]
C:\...\scrape_x> dir /b
.gitignore
config.example.json
README.md
requirements.txt
scraping.py

C:\...\scrape_x> type config.example.json
{
  "cookies_list": [
    {
      "auth_token": "ISI_AUTH_TOKEN_AKUN_1",
      "ct0": "ISI_CT0_AKUN_1"
    },
    {
      "auth_token": "ISI_AUTH_TOKEN_AKUN_2",
      "ct0": "ISI_CT0_AKUN_2"
    }
  ],
  "keywords": ["Prabowo"],
  "target_tweets": 500,
  "tweets_per_account": 250,
  "delay_between_requests": 10,
  "max_tweets_per_request": 100,
  "max_retry_on_duplicate": 5
}
```

PROBLEM

Researchers and analysts need bulk access to public posts matching specific keywords, but exporting them manually is slow and the official API is costly.

MY ROLE

Solo developer — design, implementation, and testing.

APPROACH

Prototyped with the Twikit library against a single account, then added rotation across multiple cookie sessions to survive rate limits, and standardized output formats.

KEY CHALLENGE

Rate limiting and session invalidation: long runs would die mid-collection. Handling pagination and duplicate posts across pages also needed care.

SOLUTION

Implemented automatic cookie-based account rotation — when one session hits a limit, the scraper transparently switches to the next account and continues. Results are deduplicated and exported to both CSV and JSON.

RESULT

A working command-line tool that collects keyword-matched posts continuously without manual intervention, used for personal research datasets.

WHAT I LEARNED

Practical web-data ethics and rate-limit handling, session management, and designing for failure (resumable runs, structured logging).

NEXT STEPS

Add keyword scheduling, media download support, and a simple summary report of collection statistics.

CRUD Form Sekolah

PHP

A PHP CRUD application for managing school data — the project where I first connected forms, databases, and server-side validation into one working system.

MySQL

CRUD

DATA SISWA

TAMBAH DATA

NO.	NISN	NAMA LENGKAP	ALAMAT	AKSI	
1	0045871234	Ahmad Rizki Pratama	Jl. Sudirman No. 12, Batam Center	EDIT	HAPUS
2	0045871235	Siti Nurhaliza	Jl. Ahmad Yani No. 45, Batam	EDIT	HAPUS
3	0045871236	Budi Santoso	Jl. Raya Kabil No. 8, Nongsa	EDIT	HAPUS
4	0045871237	Dewi Lestari	Jl. Imam Bonjol No. 30, Sekupang	EDIT	HAPUS
5	0045871238	Muhammad Farhan	Jl. Diponegoro No. 7, Batu Aji	EDIT	HAPUS

PROBLEM

School administrative data (students, classes, records) is often managed on paper, with no validation and no history.

MY ROLE

Solo developer — full stack, from schema to UI.

APPROACH

Designed the schema, built each CRUD operation one at a time, and hardened the forms with server-side validation after the happy paths worked.

KEY CHALLENGE

Understanding how form submissions map to database operations, preventing invalid or duplicate entries, and writing SQL safely.

SOLUTION

A straightforward PHP + MySQL application with prepared statements, server-side validation on every write operation, and clean feedback for each action.

RESULT

A functional data-management app that became the foundation for my later Laravel work — the same concepts, now with a framework.

WHAT I LEARNED

Core CRUD mechanics, SQL injection awareness (prepared statements), and why server-side validation is non-negotiable.

NEXT STEPS

Rebuild it in Laravel as a comparison exercise to show the same problem solved at a higher abstraction level.

HIGHLIGHTS & ACTIVITIES

Beyond the code.

APPLE DEVELOPER ACADEMY @ INFINITE LEARNING

Applicant, Cohort 2027 — Batam, Indonesia · 2026.
Preparing through Apple ecosystem learning paths while finishing my informatics degree.

OPEN-SOURCE PORTFOLIO ON GITHUB

github.com/sakeeeeeeeee · 2026 — web apps, automation tools, and a tested trading bot with continuous integration.

TEST-DRIVEN & CI WORKFLOWS

Self-directed · 2026 — automated test suites and GitHub Actions pipelines across my projects, including 18 feature tests on my main web app.

FULL INTERACTIVE VERSION

More screenshots, case studies, and contact — farhan-aziz-harahap.vercel.app

LET'S TALK

Thanks for reading.

Every project in this portfolio is open source — clone it, read the code, break it. I usually reply to emails within a day.

Email

ggfarhan07@gmail.com

GitHub

github.com/sakeeeeeeeee

Portfolio

farhan-aziz-harahap.vercel.app